

AGENTIC AI & AGENTIC CODING

4-Day Hands-On Programme for Application Development Teams · Instructor-Led, Classroom Training

PROGRAMME DURATION 4 Days · 32 Contact Hours	DELIVERY LOCATION Thane, Mumbai (Client Premises)	DATES As Mutually Agreed	RECOMMENDED BATCH SIZE 15–20 Participants (Max 25)
AUDIENCE Developers, Sr. Devs, Tech Leads, QA & DevOps	FORMAT ~25% Concept · 75% Hands-on Labs · Scored Capstone	PRIMARY TOOLCHAIN Claude Code · VS Code · Amazon Bedrock	DATE 11 September 2026

WHAT YOU'LL LEARN

- Explain what an agent is, name its components, and judge when an agent is — and is not — the correct solution
- Build a working tool-using agent from first principles, then in a framework, with step limits, cost budgets and full tracing
- Drive real greenfield and brownfield development work through Claude Code, with a review discipline that catches what the agent gets wrong
- Ground Claude Code in a specific codebase using CLAUDE.md, custom commands, subagents and hooks
- Design robust tool interfaces and build an MCP server that exposes internal systems to agents safely
- Wire an agent into an existing application — invocation, streaming, timeouts, fallback and persistence
- Apply retrieval and grounding concepts, and integrate whichever retrieval platform the organisation selects
- Implement a multi-agent workflow with explicit handoffs, shared state, checkpointing and human-in-the-loop approval
- Test an agentic system with a golden task set and regression evals wired into CI
- Identify and mitigate the principal security risks — prompt injection, excessive agency, data exfiltration, supply-chain compromise
- Run an agent headlessly inside a CI/CD pipeline, with guardrails on the merge path
- Instrument and trace an agentic system, and debug a failed run from its trace

PROGRAMME HIGHLIGHTS

- Built for developers who will design, integrate, test and operate agents — not for an audience making platform or rollout decisions
- Two tracks covered properly and kept distinct: Agentic Coding (AI agents doing development work) and Agentic AI (building agentic capability into shipped products)
- Concepts taught platform-agnostically so the team can integrate whichever platform the organisation selects; a single stack is used for the labs so the hands-on work stays concrete
- Labs run on the team's own stack — Java and Spring Boot on the back end, TypeScript and Angular on the front end
- Days 1–2 deliver immediate, measurable productivity value to every developer, even for teams that stop there
- Days 3–4 build the capability to ship agentic features safely — MCP integration, application integration, multi-agent orchestration, testing and security
- From Day 2 onward, labs run against the client's own repository (or a sanitised copy) — not toy to-do-list examples
- Theory is kept to what a developer needs to build correctly; each day ends with a demonstrable artefact, not a quiz
- Programme closes with a scored capstone built on the team's own problem and handed over as running code

1. PROGRAMME RATIONALE

Application development teams are being asked to adopt agentic AI on two fronts at the same time, and the two are routinely confused with each other. This programme separates them and covers both properly.

Track	What It Means For The Team
Agentic Coding	- Using AI agents to do development work: build features, write and repair tests, comprehend and refactor legacy code, migrate frameworks, review pull requests - Delivered on Claude Code in VS Code, with a short session on what transfers to other tools - The engineering discipline that has to sit around it — scoping, grounding, reviewing, measuring, and knowing when not to use it
Agentic AI	- Building agentic capability into the products the team ships: tool-using agents, MCP integrations with internal systems, agents wired into existing applications, multi-agent workflows with human oversight - The engineering concerns that decide whether it survives contact with production: testing, observability, security and reliable behaviour

The programme is built so that Days 1–2 deliver immediate, measurable productivity value to every developer in the room, while Days 3–4 build the capability to ship agentic features safely. Teams that only want the first outcome still get a complete two-day arc; teams pursuing both get a coherent four-day progression ending in a working capstone build.

2. TARGET AUDIENCE AND PREREQUISITES

Who this is for

- Application developers and senior developers (2+ years of professional experience)
- Technical leads and architects working alongside the development team
- QA and automation engineers working alongside the development team
- DevOps engineers who will operate agentic components in the pipeline

The content is pitched at the developer building and integrating agents. Organisational adoption strategy, rollout planning and governance are deliberately out of scope, and are better addressed separately with leads and architects.

Prerequisites

- Essential** — working proficiency in Java, or in TypeScript/JavaScript; comfortable with Git branching and pull requests; understands REST APIs and JSON. Comfortable working at a command line.
- Helpful** — basic cloud familiarity (roles, regions, consoles); basic Docker familiarity; exposure to CI/CD pipelines; prior use of any AI coding assistant.
- Not required** — machine learning background, data science experience, Python proficiency, or any prior model training knowledge. This is an engineering programme, not an ML programme.

3. TECHNOLOGY STACK AND TOOLCHAIN

Concepts are taught so that they transfer to whichever platform the organisation selects. A single stack is used for the labs so that the hands-on work stays concrete rather than hypothetical.

Area	Stack	Coverage
Lab languages	Java / Spring Boot and TypeScript / Angular	- Matches the team's own development stack, so lab output is directly transferable - Python is available as an alternative for participants who prefer it
Agentic coding platform	Claude Code, used with Visual Studio Code	- Authenticated against Amazon Bedrock, so no separate vendor procurement is required - Covered in depth on Day 2 and used throughout Days 3–4 - A short Day 2 session covers what transfers to other agentic coding tools
IDE	Visual Studio Code	All labs and demonstrations are built for VS Code
Model platform	Amazon Bedrock	- Runs inside the existing AWS account, so data stays within the client tenancy - Used as the concrete implementation behind platform-agnostic concepts - Model enablement and quota in the chosen region are set up before Day 1 — see Section 11
Agent framework	Claude Agent SDK, with a short introduction to LangGraph	- The Claude Agent SDK is the same harness that underlies Claude Code, so Day 4 builds directly on Day 2 - LangGraph is introduced at a basic level — what a graph and state machine buy you, and when that structure is worth it - Other options are noted briefly so the team recognises them, without a comparative deep-dive
Retrieval and vector store	Local store for labs; concepts taught platform-agnostically	- Retrieval, embeddings and grounding are taught conceptually, so the team can integrate any managed or self-hosted store - Classroom labs use a local, zero-infrastructure store so no provisioning is required - Detailed managed-service implementation is available as a separate module if required

4. FOUR-DAY STRUCTURE AT A GLANCE

Day	Theme	Core Content	Principal Lab
1	Agentic AI Foundations	Agent fundamentals, context and control, design patterns you will actually use	Agent built from scratch, no framework
2	Agentic Coding with Claude Code	Claude Code fundamentals, CLAUDE.md grounding, plan-act-review loop, portability, metrics	Greenfield build + legacy refactor
3	Tools, MCP, Integration & Security	Tool design, retrieval concepts, MCP architecture, wiring agents into applications, OWASP LLM Top 10	Build an MCP server; integrate an agent into a service
4	Multi-Agent, Testing, CI/CD & Capstone	Three orchestration patterns, Claude Agent SDK, LangGraph introduction, evals, tracing, agentic CI/CD	Multi-agent handoff, eval harness, scored capstone

Standard day: 09:30 – 17:30, with a 15-minute mid-morning break, a 45-minute lunch break and a 15-minute afternoon break. Timings are adjustable to client site conventions.

5. PROGRAM CONTENT — DETAILED DAY-WISE OUTLINE

1 Agentic AI Foundations — Building Your First Agent

FOUNDATIONS · DAY 1 · ALL PARTICIPANTS

Objective: Get to a working agent quickly. Enough concept to design one properly, then straight into building a tool-using agent from first principles.

Kick-off & Baseline	- Programme map, ground rules, team formation for the programme - Baseline skills survey and current AI-tool usage in the team - Capstone brief issued on Day 1 so teams collect candidate problems throughout - Environment check: model access, credentials and a first successful call from every machine
Agent Fundamentals	- Where agents sit relative to prompting, RAG and plain tool use — a short orientation, not a taxonomy - What "agentic" actually means: autonomy, goal decomposition, environment feedback - The six components: model, instructions, tool registry, memory, the loop, stopping conditions - Determinism vs. non-determinism: what you can and cannot promise a stakeholder - Model selection in practice: capability and cost tiers, and how to choose without over-thinking it
Context and Control	- Context engineering: window economics, ordering, compaction, output contracts (JSON Schema) - Token budgets, step caps and cost ceilings as first-class design parameters - Stopping conditions and why an agent without one is a production incident waiting to happen
LAB 1 — Build an Agent from Scratch	- A tool-calling loop written by hand, with no framework, in the team's own language - Three tools wired in: filesystem read, HTTP GET, calculator - Full step tracing so every decision the agent makes is visible

	• Deliberate failure injection: bad tool output, infinite loop, hallucinated arguments • Extended lab time — this is the session that makes everything afterwards make sense
Design Patterns You Will Actually Use	• The tool-use loop (ReAct), plan-and-execute, reflection / self-critique, and routing • Pattern-to-problem matching exercise using the team's own backlog items • When NOT to use an agent — deterministic workflows, simple RAG, plain automation • Cost, latency and reliability trade-offs, kept practical
Debrief	• Day-1 recap and troubleshooting clinic • Environment checks for Day 2

HANDS-ON LABS — DAY 1

Lab 1.1 — Bare-metal agent loop: implement the observe → decide → act → observe cycle by hand

Lab 1.2 — Tool schema design clinic: write tool descriptions an agent actually uses correctly, and measure the difference

Lab 1.3 — Failure-mode gallery: reproduce and then fix five classic agent failures (loop, drift, over-calling, silent failure, argument hallucination)

End-of-day checkpoint: Each participant demonstrates a working, traced agent loop with at least two tools and a hard step limit.

2 Agentic Coding with Claude Code — Agents Inside the Development Workflow

APPLIED PRACTICE · DAY 2 · ALL PARTICIPANTS · CLAUDE CODE + VS CODE

Objective: Drive real greenfield and brownfield work through Claude Code on the team's own stack, with a disciplined review loop and honest measurement of what is and is not being gained.

Recap & Orientation	• Day-1 recap; the four generations of coding assistance: completion → chat → agent mode → autonomous/headless agents • Where Claude Code sits architecturally: a terminal-native agent with an IDE surface, rather than an editor plug-in
Claude Code Fundamentals	• Installation, authentication and the VS Code integration • The core loop: plan mode, edit mode, and how to keep the agent inside a reviewable scope • Permission modes and tool allow-lists — deciding what the agent may do without asking • Context management in practice: what fills the window, when to compact, when to start clean • Session management, resuming work, and headless invocation for scripted tasks
Grounding Claude Code in Your Codebase	• CLAUDE.md: the memory hierarchy (enterprise, project, user), what belongs in each, and what to leave out • Custom slash commands for the team's repeated tasks; building a shared command library • Subagents: delegating narrow work to focused configurations, and when this is worth the complexity • Hooks: enforcing formatting, linting and test gates deterministically rather than by asking politely • Coding standards, architecture decisions and lint rules expressed as machine-readable context
LAB 2 — Greenfield Build	• From a written spec to a running service with tests, agent-driven, inside 60 minutes • Built on the team's own stack — Spring Boot on the back end, Angular on the front end • The plan → act → diff → review loop; scoping a task so the agent can actually finish it • One-intent-per-change discipline and reviewable diff sizes

	Comparative timing against a manual baseline the team estimates first
LAB 3 — Brownfield / Legacy Work	- Codebase comprehension: getting a trustworthy explanation of unfamiliar code - Characterisation tests first, refactor second — using tests as the guardrail - Framework and version migration assistance; dependency upgrade workflows - Bug reproduction from a stack trace and a defect description - Extended lab time — the highest-value session for a team maintaining an existing product
Portability, Discipline and Measurement	- What transfers between agentic coding tools and what does not — portable prompt and review discipline, repository context files, MCP servers, task scoping - Review discipline: what a human must always verify; the rubber-stamping trap - Anti-patterns: context flooding, agent thrash, scope creep in a single run, security-blind acceptance - Licensing and IP hygiene for generated code - Metrics that matter: suggestion acceptance, rework/revert rate, cycle time, defect escape rate

HANDS-ON LABS — DAY 2

Lab 2 — Greenfield: spec to running service with agent-generated tests and a reviewed diff, on the team's own stack

Lab 3 — Brownfield: legacy module comprehension → characterisation tests → agent-assisted refactor

Lab 3b — Grounding clinic: write the team's first real CLAUDE.md and two custom commands, then measure the change in agent behaviour

End-of-day checkpoint: Every team ships one reviewed, merged agent-assisted change to the training repository, with a written review note explaining what they accepted, rejected and why.

3 Tools, MCP, Integration and Security

TOOLS · MCP · RETRIEVAL · INTEGRATION · SECURITY · DAY 3 · ALL PARTICIPANTS

Objective: *The core integration day: design robust tools, expose internal systems through MCP, wire an agent into an existing application, and secure the result.*

Tool and Function Calling in Depth	- Schema design: naming, descriptions as prompts, parameter typing, enums over free text - How a tool definition maps to what the model actually sees — the concept, then the concrete API - Idempotency, retries and error contracts an agent can actually recover from - Read tools vs. write tools; approval gates on anything destructive - Structured outputs and validation at the boundary
Retrieval and Grounding — Concepts	- When to retrieve vs. when to call a tool — the practical decision rule - Chunking, embeddings, vector vs. keyword vs. hybrid search, reranking - What "grounding" actually requires: relevance, freshness, permissions, and citation of sources - Managed retrieval services and self-hosted stores — a short orientation, platform-agnostic - Detailed managed-service implementation is available as a separate module if required
LAB 4 — Model Context Protocol (MCP)	- Why a protocol: the N×M integration problem and how MCP solves it - Architecture: host, client, server; primitives — tools, resources, prompts; stdio and HTTP transports - Build an MCP server that exposes a mock internal API (tickets, lookup, configuration)

	• Register the same server with Claude Code and with a second client; observe the reuse first-hand • Scoping MCP servers to a project, a user or the whole team • Extended lab time — MCP is the most reusable integration skill in the programme
Integrating Agents into Existing Applications	• Calling an agent from an existing back-end service, and where that call belongs in the architecture • Synchronous vs. asynchronous invocation; queues and long-running agent work • Streaming responses through to a front end; timeouts, cancellation and partial results • Error handling and fallback when the agent fails or exceeds its budget • What to persist: run history, traces and results
Security in Practice	• Least privilege: execution roles scoped to the smallest workable permission set; sandboxing and network egress control • Secrets management; never in prompts, never in logs, never in context files • OWASP Top 10 for LLM applications, applied to the team's own designs • Direct and indirect prompt injection, excessive agency, insecure output handling, tool poisoning and confused deputy • Supply-chain risk in agent extensions, MCP servers and shared prompt libraries • Guardrail layers as enforcement; audit trails and IP hygiene for generated code

HANDS-ON LABS — DAY 3

Lab 4.1 — Tool design clinic: refactor three badly specified tools and measure the change in agent success rate

Lab 4.2 — Build a working MCP server over a mock internal API and register it with Claude Code

Lab 4.3 — Wire an agent into an existing service and surface a streamed response to a front end

Lab 4.4 — Red-team an agent: extract a secret, trigger an unauthorised tool call, then fix both

End-of-day checkpoint: Each team demonstrates an MCP server serving at least three tools to Claude Code, with an approval gate on every write operation and secrets held outside the codebase.

4 Multi-Agent Systems, Testing, CI/CD and Capstone

ORCHESTRATION · TESTING · CI/CD · CAPSTONE · DAY 4 · ALL PARTICIPANTS

Objective: Move from one agent to several under control, prove the result works, run it in a pipeline — then build and demonstrate something real on the team's own problem.

Orchestration and Frameworks	• Single agent with tools first: the correct default, and the honest test for when it stops being enough • Pattern 1 — Sequential pipeline: explicit handoffs between stages, and where it breaks • Pattern 2 — Supervisor / router: one coordinator delegating to focused workers • Pattern 3 — Evaluator / reviewer: a second agent checking the first, the highest-value pattern for development work • Claude Agent SDK: the same harness that runs Claude Code, used to build the team's own agents • LangGraph: a basic introduction — what a graph and state machine buy you, and when that structure is worth it • State, memory, checkpointing and the human approval gate, kept practical
-------------------------------------	---

Testing, Evaluation and Observability	• Outcome evaluation vs. trajectory evaluation — and why you usually need both • Building a golden task set from real team work; measuring consistency across runs • LLM-as-judge: how to use it and where it quietly misleads you • Tracing spans for agent runs; debugging a failed run from its trace alone • What to log, what to redact, and retention/PII considerations
Agentic CI/CD — Developer Implementation	• Running an agent headlessly inside a pipeline: invocation, exit codes, artefacts • Automated PR review as a pipeline stage, wired up in the lab • Regression evals inside CI; blocking a merge on an eval failure • Self-healing test suites and dependency-upgrade jobs — how they are actually built • Guardrails on the merge path: what an agent may and may not do without a human • Kept implementation-level throughout, not policy-level
CAPSTONE — Build and Demo	• Teams of 3–4 build a working agentic solution against a real problem from their own backlog • Teams may extend the multi-agent pipeline from the morning lab, or build a focused single-agent solution • Mandatory elements: a guardrail, an eval, a trace and a human approval point • Trainer floats as reviewer and unblocker throughout • 10-minute demo plus 5-minute Q&A per team, scored against the published 100-point rubric
Close	• Post-assessment, feedback and certificates • Capstone handover: each team leaves with running code and its documentation

HANDS-ON LABS — DAY 4

Lab 5.1 — Multi-agent handoff: build a two-stage pipeline with a shared state store and a human approval gate

Lab 5.2 — Eval harness: build a golden set and wire it into a CI job that blocks on failure

Lab 5.3 — Agent-assisted PR review running as a pipeline stage

Capstone — Team build, demo and scored review

End-of-day checkpoint: Capstone demo delivered and scored; each team hands over working code with its guardrail, eval and trace in place.

6. CAPSTONE PROJECT

Teams of three or four select one problem from their own backlog and build a working agentic solution during the Day 4 afternoon. Candidate problems are collected from Day 1 onward and confirmed with the trainer on the morning of Day 4, so the scope is achievable and the output has a chance of surviving beyond the classroom.

Capstone options

- Defect-to-fix agent: stack trace and defect description in; reproduction test and candidate patch out, human reviewed
- Legacy modernisation crew: comprehension, characterisation tests and a staged refactor plan for a nominated legacy module
- Internal-systems agent over MCP: an agent that safely queries and acts on the team's own APIs
- Test-coverage uplift agent: raises coverage on a target module and reports honestly on the quality of what it produced
- Requirements-to-scaffold pipeline: user story in; reviewed skeleton, API contract and test stubs out
- Agent embedded in an existing application, invoked from a service and surfaced to a front end

- Extension of the Day 4 multi-agent pipeline, applied to a real problem from the team's backlog
- Team-proposed problem, subject to trainer sign-off on Day 4 for scope feasibility

Scoring rubric — 100 points

Criterion	Points	What Is Being Assessed
Working functionality	25	It runs, and it solves the stated problem
Agent design and pattern fit	20	The chosen pattern suits the problem; no gratuitous complexity
Tool and MCP integration	15	Tool schemas, error handling, integration quality
Guardrails and security	15	Approval gates, least privilege, injection resistance, secrets hygiene
Testing and observability	10	A golden set exists; runs are traceable; failures can be diagnosed
Demo and documentation	15	Clear demo, honest limitations, handover-ready notes
Total	100	

7. ASSESSMENT, CERTIFICATION AND DELIVERABLES

Assessment

- Baseline assessment on Day 1 and a matched post-assessment on Day 4, so movement is measured rather than inferred from a satisfaction score
- Four end-of-day checkpoints, each requiring a working artefact rather than a quiz result
- Scored capstone against the published rubric above

Participant deliverables

- Complete slide decks for all four days
- A lab repository containing every exercise, with worked solutions on a separate branch
- Reference handbook: patterns, tool schema templates, a CLAUDE.md template, MCP server starter, integration patterns, security checklist, eval harness template
- A team working agreement for agentic coding, drafted on Day 2 and refined on Day 4
- Certificate of completion for each participant

8. LAB ENVIRONMENT AND PREREQUISITES

This is a hands-on programme. A separate detailed installation guide with step-by-step procedures is available on request.

Participant workstation

Item	Requirement
Machine	16 GB RAM minimum, 20 GB free disk, administrator rights to install software for the full duration
IDE	Visual Studio Code, with permission to install and update extensions
Agentic coding tool	Claude Code, installed and authenticated against Amazon Bedrock
Runtimes	Java JDK 21 and Node.js 20+, matching the team's stack; Python 3.12 optional
Build tools	Maven or Gradle as per the team's standard, and npm
Other	Git, Docker Desktop (or an equivalent container runtime)
AWS access	An IAM role or user per participant with Bedrock invoke permissions in the chosen region
Network	Outbound HTTPS to Bedrock endpoints, package registries (npm, Maven Central, PyPI), and the lab repository host
Accounts	Access to a sandbox Git repository for lab work — no production credentials at any point

Amazon Bedrock readiness

Model access in Bedrock is not enabled by default. The Claude models used in the programme must be explicitly enabled in the chosen region, and quota must be sufficient for the full batch working concurrently.

Per-participant IAM access and a connectivity test from a representative participant machine should be completed at least five working days before Day 1.

Where corporate proxy or egress restrictions apply, the required domains must be allow-listed in the same window. A one-page allow-list document is available on request.

Classroom

- Projector or large display with HDMI, whiteboard with markers, power at every seat
- Seating that allows four or five teams to work together, with room to move between them
- Stable Wi-Fi sized for the full batch working concurrently against Bedrock and package registries

9. DELIVERY METHODOLOGY

Principle	How It Shows Up In The Room
Hands-on majority	Roughly 75% of contact time is lab work. Concepts are introduced only as far as needed to start building, then deepened after the participants have hit the problem themselves

Principle	How It Shows Up In The Room
Their stack, their code	Labs are built in Java and TypeScript to match the team. From Day 2 onward, they run against the client's own repository or a sanitised copy — generic to-do-list examples are avoided
Portable concepts, concrete labs	Design, integration, testing and operating skills are taught so they transfer to any platform, while the hands-on work stays on one stack so nothing is left hypothetical
Honest about limits	Failure modes, cost and the cases where agents are the wrong answer are taught as first-class content. Adoption fails faster when a team is oversold
Artefact-gated days	Each day ends with a demonstrable artefact, not a quiz. A participant who cannot produce it gets targeted support the next morning
Mixed ability handled	Paired working, stretch tasks for fast finishers, and pre-built checkpoints so nobody is stranded by a broken environment
Live coding, not slide reading	Every major concept is demonstrated live, including the mistakes and the recovery from them